POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Numerical Analysis for Machine Learning Project

Numerical Analysis of PageRank

Sparse Computation and Applications to Citation Networks

Author: Riccardo Infascelli (Codice Persona: 10742969, Matricola: 259426)

Professor: Edie Miglio

Academic year: 2025–2026

---

1 Introduction

PageRank is a link-analysis algorithm introduced by Larry Page and Sergey Brin to estimate the importance of webpages from the hyperlink structure of the Web. The Web is represented as a directed graph, where webpages are nodes and hyperlinks are edges. The underlying idea is that a page is important when it is linked by other important pages.

In this project, PageRank is implemented in Python and studied from a numerical perspective, with particular attention to the influence of its main parameters and to the role of sparse computation. The algorithm is then applied to a real citation network, where PageRank is compared with citation count to study how the structure of citations affects the ranking of scientific papers. Finally, a dynamic update experiment is used to investigate how previous PageRank computations can help reduce the cost of recomputing the ranking after changes in the graph.

The source code, notebook, and project material are available on GitHub at `github.com/RiccardoInfascelli/numerical-analysis-pagerank`.

The mathematical formulation follows the framework presented by Langville and Meyer in *Google's PageRank and Beyond* [2].

The Web as a document collection. Langville and Meyer describe the Web as a document collection with four distinctive properties: it is huge, dynamic, self-organized, and hyperlinked. The first three characteristics make web information retrieval particularly difficult. The large scale of the Web creates major storage and computational requirements, its content and structure change continuously, and the absence of centralized organization leads to heterogeneous data, broken links, and potentially unreliable information.

The hyperlink structure, however, provides additional information that is not available in traditional document collections. A hyperlink can be interpreted as an implicit recommendation from one page to another. Link-analysis methods such as PageRank exploit this structure to estimate the global importance of webpages and to improve the ordering of search results [2].

Role in web search. In a classical web-search architecture, a crawler automatically explores the Web, downloads webpages, and follows their hyperlinks to discover new pages. The collected information is then processed by

an indexing module, which stores both the textual content of the pages and the structure of the links connecting them.

When a user submits a query, the textual index is used to identify pages that are relevant to the requested terms and to assign them a query-dependent content score. PageRank plays a different role: it provides a global popularity score based only on the hyperlink structure of the Web. Since this score does not depend on a particular query, it can be computed offline, stored, and reused for many searches. At query time, the precomputed PageRank score can therefore be combined with the content score to obtain an overall ranking of the relevant pages [2].

2 Mathematical Formulation

2.1 Web Graph and Hyperlink Matrix

A collection of webpages can be represented as a directed graph

$$\mathcal{G} = (V, E),$$

where each node represents a webpage and a directed edge $i \to j$ indicates that page i contains a hyperlink to page j.

The basic idea of PageRank is recursive: a page is important when it is linked by other important pages. Each page distributes its current score equally among its outgoing links. Therefore, before introducing dangling nodes and teleportation, the score of node i can be expressed as

$$x_i = \sum_{j \to i} \frac{x_j}{\text{outdeg}(j)},$$

where $\text{outdeg}(j)$ is the number of outgoing links from node j.

This relation shows that PageRank scores cannot be computed independently. The score of each page depends on the scores of the pages linking to it, which in turn depend on the scores of other pages. PageRank therefore searches for a vector of scores that is consistent with the propagation of rank through the entire graph.

The graph structure can be represented by the hyperlink matrix $H \in \mathbb{R}^{n \times n}$, defined as

$$H_{ij} = \begin{cases} \dfrac{1}{\text{outdeg}(i)}, & \text{if } i \to j, \\ 0, & \text{otherwise.} \end{cases}$$

Each nonzero entry H_{ij} represents the fraction of the score of page i transferred to page j. Since each page divides its score equally among its outgoing links, every non-dangling row of H sums to one.

Throughout this report, PageRank vectors are treated as column vectors, consistently with the Python implementation. Since H is normalized by rows, one rank-propagation step is written as

$$x^{(k+1)} = H^T x^{(k)}.$$

At iteration k, each page j distributes its current score $x_j^{(k)}$ among the pages to which it points. Consequently, the i-th component of $H^T x^{(k)}$ is the total score received by page i from all pages linking to it:

$$\left(H^T x^{(k)}\right)_i = \sum_{j \to i} \frac{x_j^{(k)}}{\text{outdeg}(j)}.$$

Repeated application of H^T propagates the scores through the graph. However, pages without outgoing links produce zero rows in H, so the total rank may not be preserved. Moreover, the hyperlink structure alone does not always guarantee a unique convergent ranking. These problems are addressed through dangling-node correction and teleportation in the following subsection.

2.2 Dangling Nodes and Teleportation

The hyperlink matrix H describes how rank is transferred through the actual links of the graph. However, the raw hyperlink structure does not necessarily define a valid and well-behaved Markov chain. Two modifications are required before the PageRank vector can be computed reliably.

Dangling nodes. A node with no outgoing links is called a *dangling node*. In this case, the corresponding row of H is zero. From the random surfer interpretation, the problem is immediate: after reaching such a page, the surfer has no link to follow.

The same issue appears algebraically. Since the zero row does not redistribute the rank received by the dangling node, the sum of the components of $H^T x^{(k)}$ may become smaller than one. Therefore, H is not necessarily stochastic and the PageRank vector would no longer remain a probability distribution.

Let $a \in \mathbb{R}^n$ be the dangling-node indicator vector, defined by

$$a_i = \begin{cases} 1, & \text{if node } i \text{ is dangling,} \\ 0, & \text{otherwise.} \end{cases}$$

The zero rows of H are replaced by a probability distribution v^T, producing

$$S = H + av^T.$$

The matrix S is stochastic: each non-dangling row preserves the original link distribution, while each dangling row is replaced by v^T. In this project, the uniform distribution is used,

$$v = \frac{1}{n}\mathbf{1}.$$

Thus, when the surfer reaches a dangling page, the next page is chosen uniformly from the entire graph.

Teleportation. The dangling-node correction guarantees stochasticity, but this alone does not guarantee that the PageRank vector is unique or that the iterative method converges.

For example, the graph may contain a closed group of pages with no links leading outside it. Once the random surfer enters such a group, it cannot leave, and the group behaves as a *rank sink*. More generally, if the graph is not strongly connected, different regions may support different stationary distributions, and the resulting ranking may depend on the initial vector.

Directed cycles may also produce periodic behavior. In a two-node cycle, for example, a probability distribution initially concentrated on one node alternates between the two nodes instead of converging.

To avoid these problems, PageRank introduces a small probability of moving independently of the hyperlink structure. The resulting Google matrix is

$$G = \alpha S + (1-\alpha)\mathbf{1}v^T, \qquad 0 < \alpha < 1.$$

With probability α, the random surfer follows the links described by S. With probability $1-\alpha$, it teleports to a page selected according to v.

If v is positive, teleportation makes every page reachable from every other page and removes compulsory periodic behavior. Therefore, G is stochastic, irreducible, and aperiodic. These properties guarantee the existence of a unique positive stationary distribution and the convergence of the power method to the PageRank vector [2].

2.3 Stationary Distribution and Eigenvector Interpretation

The Google matrix can be interpreted as the transition matrix of a Markov chain describing the behavior of a random web surfer. If $x^{(k)}$ is the probability distribution of the surfer after k transitions, then, using the column-vector convention adopted in this report,

$$x^{(k+1)} = G^T x^{(k)}.$$

A stationary distribution is reached when the probability vector no longer changes after a transition. The PageRank vector x therefore satisfies

$$x = G^T x, \qquad x_i \geq 0, \qquad \sum_{i=1}^{n} x_i = 1.$$

Thus, x is a right eigenvector of G^T associated with the eigenvalue 1. Its component x_i represents the long-run probability that the random surfer is located at page i. Pages that are visited more frequently receive a larger PageRank value and are therefore considered more important.

Since the teleportation vector is positive, the Google matrix G is positive and stochastic. Therefore, the PageRank vector is unique and positive. Moreover, the power method converges to this vector from any initial probability distribution [2]. This interpretation connects PageRank with graph theory, Markov chains, and the Perron–Frobenius theory for non-negative matrices [2, 3].

3 Numerical Computation

3.1 Power Iteration and Stopping Criterion

The PageRank vector is the dominant eigenvector of G^T associated with the eigenvalue 1. Since the Google matrix is positive and stochastic, this eigenvector can be approximated using the power method [3]. Starting from an initial probability vector $x^{(0)}$, the iteration is

$$x^{(k+1)} = G^T x^{(k)}.$$

In this project, the initial vector is chosen as the uniform distribution

$$x^{(0)} = \frac{1}{n}\mathbf{1}.$$

In exact arithmetic, every iterate remains nonnegative and normalized:

$$x_i^{(k)} \geq 0, \qquad \sum_{i=1}^{n} x_i^{(k)} = 1.$$

The iteration is stopped when the difference between two consecutive vectors is sufficiently small:

$$\left\| x^{(k+1)} - x^{(k)} \right\|_1 < \varepsilon,$$

where ε is a prescribed tolerance. Since $x^{(k+1)} = G^T x^{(k)}$, this quantity can also be interpreted as the fixed-point residual

$$\left\| G^T x^{(k)} - x^{(k)} \right\|_1 .$$

The L^1 norm is a natural choice because the iterates represent probability distributions. A maximum number of iterations is also specified as a safeguard in case the required tolerance is not reached.

Although the iteration can be written using the Google matrix G, forming this matrix explicitly is impractical for large graphs because the teleportation term makes it dense. The implementation therefore uses an equivalent matrix-free formulation, introduced in Section 3.2.

3.2 Sparse and Matrix-Free Implementation

Although the power iteration is formally written as

$$x^{(k+1)} = G^T x^{(k)},$$

the Google matrix should not be constructed explicitly. Indeed, the teleportation term connects every node to every other node, making G dense even when the original graph is sparse. Explicit storage would therefore require $O(n^2)$ memory, and each dense matrix-vector product would require $O(n^2)$ operations.

In contrast, the hyperlink matrix H contains one nonzero entry for each directed edge. If the graph has n nodes and m edges, then

$$\mathrm{nnz}(H) = m,$$

which is typically much smaller than n^2. The matrix H can therefore be stored efficiently in a compressed sparse format.

The dangling-node and teleportation corrections are generally dense. However, both have rank-one structure and do not need to be stored as matrices. Their action on a vector can be evaluated as

$$(av^T)^T x^{(k)} = v\left(a^T x^{(k)}\right)$$

and

$$(\mathbf{1}v^T)^T x^{(k)} = v\left(\mathbf{1}^T x^{(k)}\right).$$

Since $x^{(k)}$ is a probability vector, $\mathbf{1}^T x^{(k)} = 1$. Therefore, the dense corrections reduce to scalar-vector products, requiring only $O(n)$ operations and without introducing any dense $n \times n$ matrix.

Using the definitions of the stochastic hyperlink matrix and the Google matrix, the power iteration can be expanded as

$$x^{(k+1)} = \alpha H^T x^{(k)} + \left(\alpha a^T x^{(k)} + 1 - \alpha\right) v.$$

The scalar

$$a^T x^{(k)}$$

is the total probability mass currently assigned to dangling nodes. It can be computed directly without explicitly constructing the corrected matrix S.

For the uniform teleportation vector

$$v = \frac{1}{n}\mathbf{1},$$

the iteration becomes

$$x^{(k+1)} = \alpha H^T x^{(k)} + \frac{\alpha a^T x^{(k)} + 1 - \alpha}{n}\mathbf{1}.$$

This formula contains three contributions: the rank transferred through the actual hyperlinks, the rank redistributed from dangling nodes, and the uniform teleportation term. Therefore, neither S nor G needs to be formed explicitly.

In the Python implementation, H is stored as a sparse matrix and the link contribution is computed through a sparse matrix-vector product. Two separate vectors are used for consecutive iterations, so that all updated components are computed from the same previous iterate.

The cost of one iteration is $O(m+n)$: $O(m)$ operations are required for the sparse matrix-vector product, while the dangling mass and the remaining vector operations require $O(n)$ work. The total memory requirement is also $O(m+n)$, making this formulation suitable for substantially larger graphs [2].

Algorithm 1 summarizes the matrix-free power iteration used in the numerical experiments.

Algorithm 1 Matrix-free power iteration for PageRank

Require: Sparse hyperlink matrix H, dangling-node indicator a, damping factor α, tolerance ε, maximum number of iterations K

Ensure: Approximation of the PageRank vector x

$n \leftarrow$ number of nodes
$x \leftarrow \frac{1}{n}\mathbf{1}$
for $k = 0, \ldots, K-1$ **do**
    $s \leftarrow a^T x$
    $y \leftarrow \alpha H^T x + \frac{\alpha s + 1 - \alpha}{n}\mathbf{1}$
    $r \leftarrow \|y - x\|_1$
    $x \leftarrow y$
    if $r < \varepsilon$ **then**
        return x
    end if
end for
return x

3.3 Convergence and the Damping Factor

The convergence of the power iteration follows from the spectral properties of the Google matrix. Since G is stochastic, its spectral radius is equal to one and

$$\lambda_1 = 1$$

is an eigenvalue. Moreover, if the teleportation vector v is positive and $0 < \alpha < 1$, then every entry of G is positive.

By the Perron–Frobenius theorem, the eigenvalue $\lambda_1 = 1$ is simple and dominant, and its associated normalized eigenvector is positive and unique. Consequently, the power iteration converges to the PageRank vector from any initial probability distribution.

The asymptotic convergence rate depends on the subdominant eigenvalue of G. In particular, the eigenvalues different from $\lambda_1 = 1$ satisfy

$$|\lambda_i(G)| \leq \alpha, \qquad i = 2, \ldots, n.$$

Therefore, the error is asymptotically reduced at a rate controlled by

$$|\lambda_2(G)|,$$

which is bounded above by the damping factor α.

The parameter α therefore has both a modelling and a numerical interpretation. Large values of α assign more importance to the actual hyperlink structure, since the random surfer follows links more frequently. However, as α approaches one, the spectral gap may become smaller and the power method generally requires more iterations. Smaller values increase the influence of teleportation and usually lead to faster convergence, but produce a ranking that depends less strongly on the graph structure.

The commonly used value

$$\alpha = 0.85$$

represents a practical compromise between these two effects. The influence of α on both the convergence speed and the resulting ranking will be investigated numerically in the experimental section 4.

3.4 Linear-System Formulation

PageRank can also be formulated as the solution of a linear system. Starting from the stationary equation

$$x = G^T x$$

and recalling that

$$G = \alpha S + (1-\alpha)\mathbf{1}v^T,$$

we obtain

$$x = \alpha S^T x + (1-\alpha)v\left(\mathbf{1}^T x\right).$$

Since x is a probability vector,

$$\mathbf{1}^T x = 1,$$

and therefore

$$x = \alpha S^T x + (1-\alpha)v.$$

Rearranging the terms gives the linear system

$$\left(I - \alpha S^T\right) x = (1-\alpha)v.$$

Since S is stochastic, its spectral radius is equal to one. Because $0 < \alpha < 1$, the matrix $I - \alpha S^T$ is nonsingular, and the system admits a unique solution.

Using the dangling-node correction

$$S = H + av^T,$$

the coefficient matrix can be written as

$$I - \alpha S^T = I - \alpha H^T - \alpha v a^T.$$

The last term is a rank-one correction and therefore does not require the explicit construction of a dense matrix.

In this project, the linear system is solved on small graphs using a sparse direct solver. The resulting vector is normalized to compensate for possible floating-point round-off:

$$x \leftarrow \frac{x}{\mathbf{1}^T x}.$$

This formulation provides an independent validation of the power iteration. The two methods should produce the same normalized PageRank vector up to numerical errors, which can be measured through

$$\left\| x_{\text{power}} - x_{\text{linear}} \right\|_1.$$

For large-scale problems, however, the power method is generally more practical because it requires only sparse matrix-vector products. Direct solution of the linear system may require expensive matrix factorizations and can generate additional nonzero entries during the factorization process.

4 Numerical Experiments and Results

The numerical experiments were carried out in Python using NumPy and SciPy. The analysis follows the same progression as the accompanying notebook: the implementation is first validated on a small reference graph, then tested in terms of convergence and computational efficiency, and then applied to a real citation network. The final experiment studies a dynamic PageRank update, comparing different initializations for recomputing PageRank after new nodes and links are added to a graph.

4.1 Implementation Validation on a Six-Page Graph

Before considering larger networks, the implementation was tested on a small directed graph with six nodes. This example provides a controlled setting in which the PageRank vector obtained through the power iteration can be compared with direct numerical formulations. It also includes a dangling node, allowing the corresponding correction to be verified explicitly.

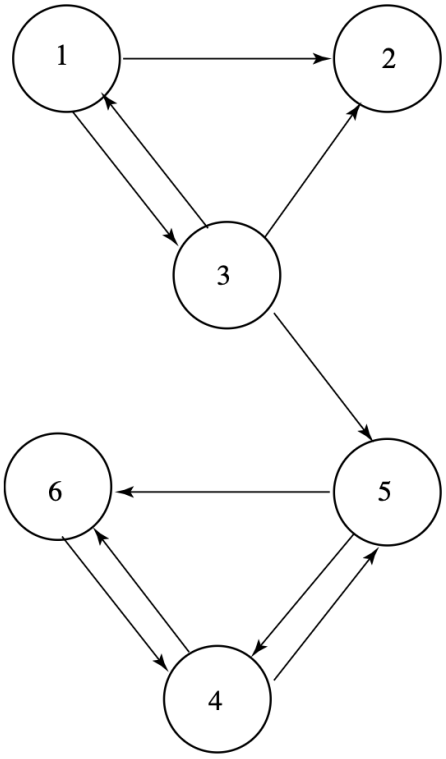

Figure 1: Six-page directed graph used for the implementation validation. Page P_2 is a dangling node.

The PageRank vector was computed using a uniform initial distribution, a damping factor $\alpha = 0.85$, and the stopping tolerance

$$\left\| x^{(k+1)} - x^{(k)} \right\|_1 < 10^{-8}.$$

Under these conditions, the power iteration converged in 33 iterations.

The computed PageRank vector is

$$x = \begin{bmatrix} 0.051705 & 0.073679 & 0.057412 & 0.348704 & 0.199904 & 0.268596 \end{bmatrix}^T.$$

The highest score is assigned to page P_4, followed by P_6 and P_5, which form the most strongly interconnected part of the graph.

For an independent validation, the Google matrix was explicitly formed and the eigenvector associated with the eigenvalue 1 was computed using a direct eigensolver. The difference between the two normalized solutions was

$$\|x_{\text{power}} - x_{\text{eig}}\|_1 = 8.54 \times 10^{-9}.$$

The difference is consistent with the prescribed stopping tolerance, confirming that the power iteration correctly approximates the stationary PageRank vector.

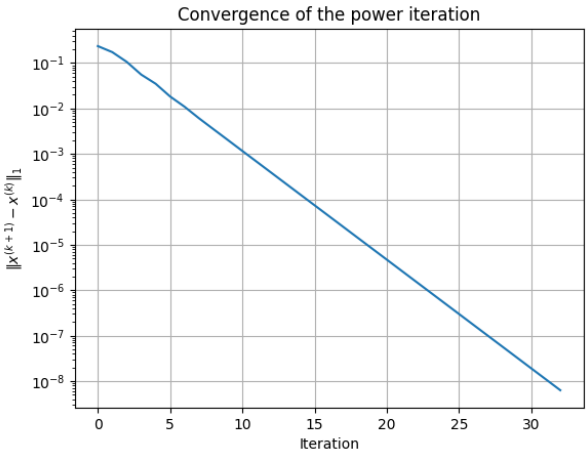

Figure 2: L^1 residual of the power iteration on the six-page graph.

The residual decreases approximately linearly on the logarithmic scale, showing the expected geometric convergence of the power iteration.

4.2 Influence of the Damping Factor

The influence of the damping factor was first examined on the six-page reference graph. The power iteration was repeated for

$$\alpha \in \{0.10, 0.50, 0.85, 0.95, 0.99\},$$

using the same uniform initial vector and stopping tolerance 10^{-8}.

The number of iterations increased as the damping factor approached one. In particular, convergence required only 7 iterations for $\alpha = 0.10$, 33 iterations for the standard value $\alpha = 0.85$, and 45 iterations for $\alpha = 0.99$.

This behaviour is consistent with the spectral analysis of the Google matrix. As α increases, the subdominant eigenvalues may move closer in magnitude to the dominant eigenvalue 1, reducing the spectral gap and slowing the convergence of the power iteration.

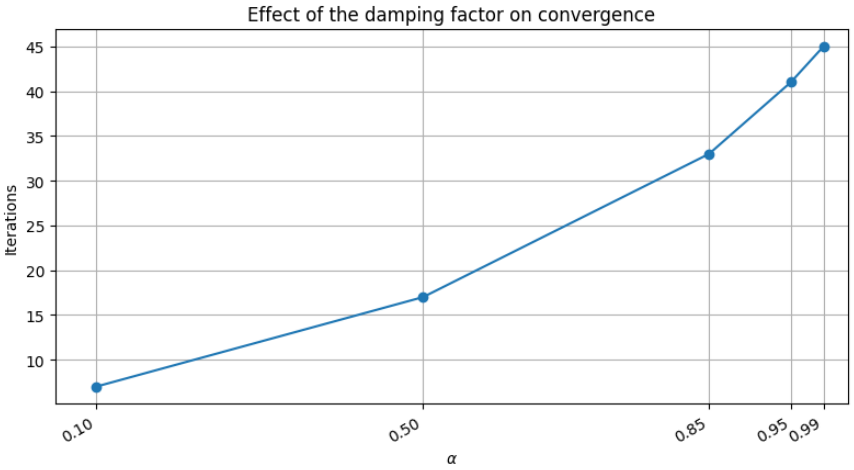

Figure 3: Number of power iterations required for different values of the damping factor on the six-page graph.

Figure 3 confirms the rapid increase in computational effort for values of α close to one.

Although the numerical values of the PageRank scores change with α, the ordering of the six pages remains unchanged:

$$P_4 > P_6 > P_5 > P_2 > P_3 > P_1.$$

For larger values of α, the scores become increasingly concentrated on pages P_4, P_5, and P_6, since this group forms the most strongly connected region of the graph.

Therefore, on this example, the damping factor has a stronger effect on the convergence rate and on the magnitude of the scores than on the final ranking order.

4.3 Sparse Implementation and Computational Performance

The matrix-free sparse implementation was first compared with the basic dense formulation on the six-page graph. Both methods converged in 33 iterations and produced PageRank vectors differing by only

$$\|x_{\text{dense}} - x_{\text{sparse}}\|_1 = 1.53 \times 10^{-16}.$$

This difference is at the level of floating-point roundoff and confirms that the sparse formulation is numerically equivalent to the dense one.

To evaluate computational performance, the two implementations were applied to randomly generated directed graphs with

$$n \in \{100, 500, 1000, 5000\}$$

nodes. Each graph had an average out-degree of approximately 10, so that the number of edges increased linearly with the number of nodes. The same damping factor, stopping tolerance, and initial vector were used for both implementations.

The execution times were measured in the same computational environment and should therefore be interpreted mainly as a relative comparison between the dense and sparse approaches.

Table 1: Execution times of the dense and sparse PageRank implementations on randomly generated graphs.

Nodes	Edges	Dense time (s)	Sparse time (s)	Speedup
100	990	0.0205	0.0075	2.72
500	4995	0.0519	0.0030	17.19
1000	9987	0.1072	0.0045	23.74
5000	49983	0.5713	0.0146	39.10

The advantage of the sparse implementation becomes increasingly significant as the graph size grows. For the largest graph, the matrix-free method was approximately 39 times faster than the dense implementation. In all experiments, the L^1 difference between the two PageRank vectors remained below 2×10^{-16}.

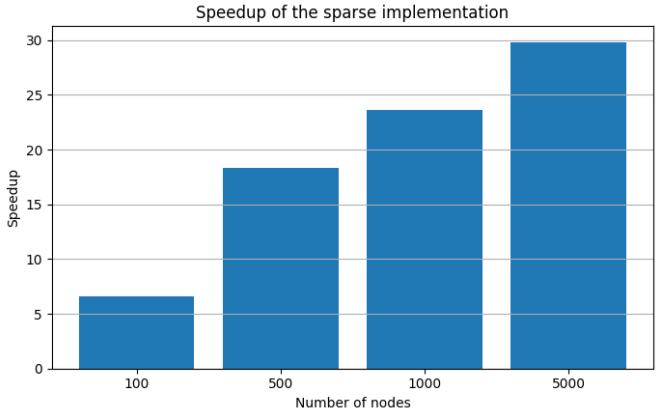

Figure 4: Speedup of the sparse matrix-free implementation with respect to the dense formulation.

Figure 4 shows that the computational advantage of the sparse implementation increases rapidly with the graph size. This result reflects the fact that the sparse method processes only the existing edges, whereas the dense formulation performs operations on the entire $n \times n$ matrix.

4.4 Comparison with the Linear-System Formulation

As an additional validation, the PageRank vector of the six-page graph was also computed by solving the equivalent linear system introduced in the previous section. For $\alpha = 0.85$, the normalized solution was compared with the vector obtained through the power iteration.

The difference between the two solutions was

$$\|x_{\text{power}} - x_{\text{linear}}\|_1 = 8.54 \times 10^{-9}.$$

The two formulations therefore agree within the prescribed iterative tolerance.

The linear-system formulation provides an independent verification of the implementation. However, directly solving the system is not appropriate for large networks, where the sparse matrix-free power iteration requires substantially less memory and computational work.

4.5 Application to the HEP-TH Citation Network

To evaluate the algorithm on a real sparse network, PageRank was applied to the High Energy Physics Theory citation dataset from the Stanford Network Analysis Project. The experiment was inspired by the analysis proposed by Chen et al. [1], although a different citation network and a simplified selection criterion were used. In this network, each node represents a scientific paper, while a directed edge from paper i to paper j indicates that paper i cites paper j.

4.5.1 Dataset and Preprocessing

The HEP-TH citation network was obtained from the Stanford Network Analysis Project [4]. The original dataset contains 27 770 papers and 352 807 citation links. Duplicate edges and self-citations were removed before constructing the network. After preprocessing, the graph contained 27 769 nodes and 352 768 directed edges.

The direction of each edge was preserved according to the citation relation: an edge $i \to j$ means that paper i cites paper j. Consequently, a paper receives PageRank contributions from the papers that cite it. The cleaned network was stored as a sparse matrix to avoid forming a dense $27\,769 \times 27\,769$ matrix.

4.5.2 PageRank Computation

The PageRank vector was computed with the sparse matrix-free power iteration using a damping factor $\alpha = 0.50$, a uniform initial distribution, and the stopping tolerance

$$\left\| x^{(k+1)} - x^{(k)} \right\|_1 < 10^{-8}.$$

The method converged in 20 iterations, with final residual

$$5.59 \times 10^{-9}.$$

The resulting vector was normalized so that its entries sum to one.

4.5.3 PageRank and Citation Count

To compare PageRank with a simpler bibliometric measure, the number of incoming citations was computed for each paper. The relationship between the two rankings was evaluated using Spearman's rank correlation coefficient.

The resulting correlation was

$$\rho_{\text{S}} = 0.868.$$

This value indicates a strong positive association: papers with many citations generally tend to receive high PageRank scores. However, the correlation is not perfect, since PageRank also accounts for the importance of the citing papers and for the complete structure of the citation network.

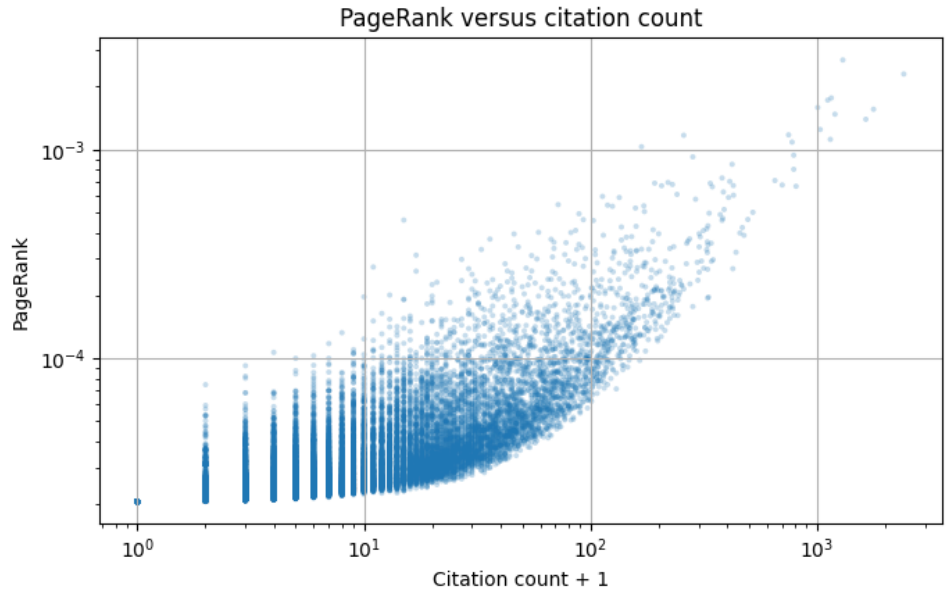

Figure 5: Relationship between citation count and PageRank in the HEP-TH network. Both axes are shown on a logarithmic scale; one is added to the citation count to include uncited papers.

Figure 5 confirms the strong positive association between citation count and PageRank. However, papers with the same number of citations can receive noticeably different PageRank scores, since the algorithm also accounts for the structural importance of the citing papers.

The differences between the two rankings are therefore particularly interesting, since they may reveal papers whose structural importance is not captured by citation count alone.

Table 2: Top ten papers in the HEP-TH network according to PageRank, with $\alpha = 0.50$.

PR rank	Paper ID	PageRank	Citations	Citation rank
1	9407087	0.002686	1299	4
2	9711200	0.002301	2414	1
3	9510017	0.001766	1155	6
4	9503124	0.001725	1114	8
5	9408099	0.001589	1006	10
6	9802150	0.001559	1775	2
7	9610043	0.001477	1199	5
8	9802109	0.001396	1641	3
9	9906064	0.001247	1032	9
10	9410167	0.001175	748	15

The PageRank ordering is similar to, but not identical to, the ranking based on citation count. For example, paper 9407087 is ranked first by PageRank despite being fourth by citation count, whereas paper 9711200, the most cited paper in the dataset, occupies the second PageRank position. This difference shows that the location of citations within the network can influence the ranking beyond their total number.

4.5.4 Influence of Publication Year

Since citations accumulate over time, older papers may have a structural advantage in the citation network. To investigate this effect, the papers were grouped according to their publication year, and both the average and median PageRank were computed for each group.

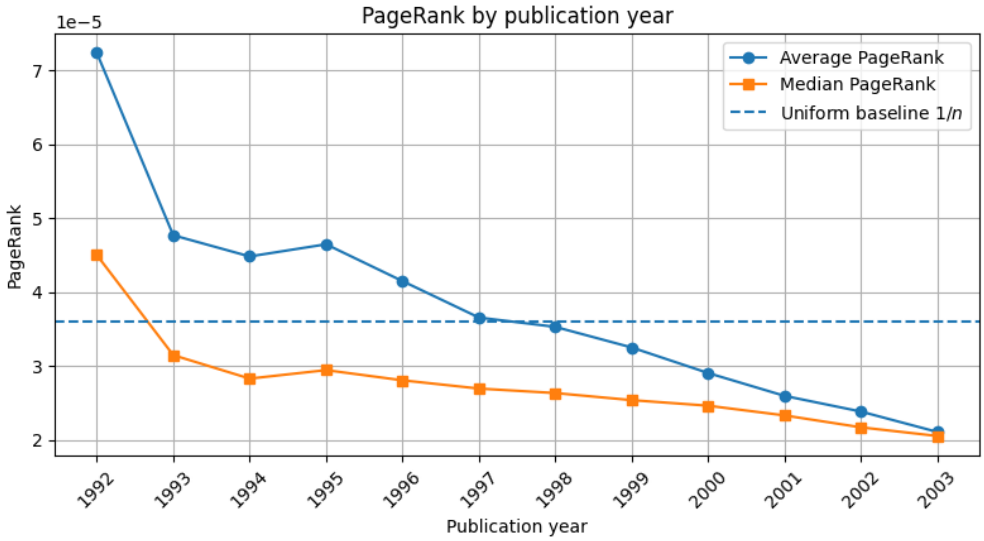

Figure 6: Average and median PageRank by publication year in the HEP-TH network. The dashed line represents the global average PageRank $1/n$.

Figure 6 shows a clear decreasing trend in both the average and median PageRank as the publication year increases. Papers published in the earliest years generally receive PageRank scores above the global average, whereas papers from the most recent years tend to fall below it.

This result suggests the presence of an age advantage in the citation network. Older papers have had more time to accumulate citations and to become connected to several generations of subsequent publications. However, the effect should not be interpreted as evidence that PageRank intrinsically favours older nodes, since it is also influenced by the finite observation window of the dataset. In particular, the year 2003 contains fewer papers and provides only a limited citation window.

The visual trend is also confirmed by the Spearman correlation between publication year and PageRank:

$$\rho_{\mathrm{S}} = -0.395, \qquad p < 10^{-16}.$$

This indicates a statistically significant moderate negative association: more recent papers tend to receive lower PageRank scores. Nevertheless, this relationship should not be interpreted as causal, since publication year is also related to the amount of time available to accumulate citations and to the finite observation window of the dataset.

4.5.5 Candidate Scientific Gems

To identify papers whose structural importance may be underestimated by citation count, the PageRank ranking was compared with the ranking based on incoming citations. This analysis follows the general idea proposed by Chen et al. [1], although the present experiment uses the HEP-TH network rather than the original APS dataset.

For each paper i, the rank ratio was defined as

$$R_i = \frac{r_i^{\mathrm{cit}}}{r_i^{\mathrm{PR}}},$$

where r_i^{cit} is its citation-count rank and r_i^{PR} is its PageRank rank.

A paper was classified as a candidate scientific gem if it belonged to the top 100 papers according to PageRank and satisfied

$$R_i > 10.$$

Using this criterion, 13 candidate papers were identified.

Table 3: Candidate scientific gems in the HEP-TH citation network.

Paper ID	PR rank	Citations	Citation rank	Rank ratio
9201015	52	14	5722	110.04
9206047	75	35	2187	29.16
9208055	97	32	2429	25.04
9205037	68	45	1546	22.74
9204102	38	71	781	20.55
9402005	63	58	1090	17.30
9201059	89	47	1450	16.29
9205068	14	167	174	12.43
9202057	47	88	558	11.87
9206007	76	66	890	11.71
9201061	34	112	368	10.82
9203056	81	68	843	10.41
9308139	51	92	521	10.22

The largest discrepancy is observed for paper 9201015, which is ranked 52nd by PageRank but only 5722nd by citation count. More generally, these papers receive relatively few citations, but the position of those citations within the network gives them a substantially higher PageRank ranking.

Inspection of the metadata shows that the candidates include papers on string theory, black holes, supersymmetry, and gravitational models. For example, paper 9201015 is entitled *An Algorithm to Generate Classical Solutions for String Effective Action*, while paper 9204102 is *A New Supersymmetric Index*. Their high PageRank position suggests that they are cited by papers that are themselves influential within the network.

However, the list also contains an erratum and an addendum. This shows that a large rank ratio should not be interpreted automatically as a measure of scientific quality. PageRank identifies structural importance in the citation network, while the scientific relevance of a paper still requires domain-specific evaluation. For this reason, the selected papers are referred to as candidate scientific gems rather than definitive scientific gems.

4.5.6 Sensitivity to the Damping Factor

The sensitivity analysis was repeated on the HEP-TH citation network for

$$\alpha \in \{0.10, 0.50, 0.85, 0.95\}.$$

For each value, the PageRank vector was computed using the same uniform initial distribution and stopping tolerance 10^{-8}.

The ranking obtained with $\alpha = 0.50$, used in the previous experiments, was taken as the reference ranking.

Table 4: Sensitivity of PageRank to the damping factor on the HEP-TH citation network. Spearman correlations are computed with respect to the ranking obtained for $\alpha = 0.50$.

α	Iterations	Final residual	Spearman correlation
0.10	7	2.52×10^{-9}	0.993961
0.50	20	5.59×10^{-9}	1.000000
0.85	81	9.41×10^{-9}	0.995629
0.95	247	9.59×10^{-9}	0.992775

The convergence rate is strongly affected by the damping factor. The number of iterations increases from 7 for $\alpha = 0.10$ to 247 for $\alpha = 0.95$, confirming that values close to one lead to substantially slower convergence.

Despite this difference in computational cost, all Spearman correlations remain above 0.992. The global ordering of the papers is therefore remarkably stable with respect to the damping factor, although local changes in rank may still occur.

These results show that, on the HEP-TH network, the damping factor has a much stronger influence on convergence speed than on the overall ranking.

4.6 Dynamic PageRank Update Experiment

Real graphs are often dynamic: new nodes and new links may be added over time. In this case, recomputing PageRank from scratch can be inefficient, especially when the updated graph is large. To study this setting, a synthetic directed graph was generated in two stages. First, an old graph was constructed. Then, new nodes and links were added to obtain the updated graph.

The experiment compares three exact strategies for computing PageRank on the updated graph. The cold-start method starts the power iteration from the uniform distribution. The warm-start method reuses the PageRank vector computed on the old graph as an initial approximation. Finally, the aggregation-assisted refinement first solves a reduced PageRank problem and then uses the resulting vector as the initial point for a full PageRank iteration on the updated graph.

In the reduced problem, all new nodes and the most important old nodes are kept explicitly, while the remaining old nodes are compressed into a single aggregate node. The aggregated solution is not used as a final PageRank vector; it is only used to initialize the refinement step on the full graph.

The three methods were compared on the updated graph. The cold-start solution was used as the reference solution, and the other methods were evaluated in terms of convergence speed and agreement with this reference.

Table 5: Comparison of PageRank update strategies on the updated graph.

Method	Iterations	Time (s)	L^1 difference
Cold start	73	1.262	0
Warm start	55	0.907	2.43×10^{-8}
Aggregation + refinement	23	0.408	2.87×10^{-8}

The warm start reduces the number of iterations because it uses information from the old graph. The aggregation-assisted refinement gives the largest reduction, since the reduced problem provides an even better initial vector for the final PageRank iteration on the full graph.

The small L^1 differences show that the three methods recover the same PageRank vector up to the prescribed tolerance. Therefore, the update strategies affect the computational cost, but not the final PageRank model.

5 Discussion

The numerical experiments confirm that PageRank can be computed accurately and efficiently through the sparse matrix-free power iteration. The tests also highlight the main numerical trade-offs of the method: the damping factor affects convergence, sparse storage is essential for large networks, and the resulting ranking contains structural information that is not captured by node degree alone.

Accuracy and consistency of the implementation. The agreement between the power iteration, the direct eigensolver, and the linear-system formulation provides strong evidence for the correctness of the implementation. In all comparisons, the differences between the computed PageRank vectors were compatible with the chosen stopping tolerance or with floating-point roundoff. Moreover, the dense and sparse implementations produced numerically equivalent results, showing that the matrix-free reformulation improves efficiency without altering the mathematical problem.

Sparsity and scalability. The performance experiments show that exploiting the sparse structure of the graph is essential as the network size increases. Since each iteration of the matrix-free method processes only the nonzero entries of the hyperlink matrix, its computational cost grows approximately with the number of nodes and edges rather than with n^2. The increasing speedup observed in the random-graph experiments confirms that this advantage becomes more significant for larger networks. This behaviour is particularly relevant for the HEP-TH dataset, where a dense representation would require storing hundreds of millions of matrix entries, most of which would be zero.

Dynamic PageRank updates. The dynamic update experiment shows that previous computations can also be useful when the graph changes. Starting the power iteration from an informed vector reduces the number of iterations required on the updated graph. In particular, the warm start uses the old PageRank vector, while the

aggregation-assisted refinement uses a reduced problem to construct an even better initial approximation. In both cases, the PageRank model is unchanged: the improvement concerns the computational cost of reaching the same fixed point.

Influence of the damping factor. The experiments confirm the expected trade-off associated with the damping factor. Larger values of α place greater emphasis on the link structure of the graph, but they also slow the convergence of the power iteration by reducing the effective spectral gap. This effect is clearly visible both on the six-page graph and on the HEP-TH network. At the same time, the high Spearman correlations observed on HEP-TH indicate that the global ranking remains relatively stable over a wide range of parameter values. Therefore, the choice of α has a stronger impact on computational cost than on the overall ordering of the papers in this experiment.

PageRank and citation-based importance. The comparison with citation count shows that PageRank captures a different notion of importance. Although the two rankings are strongly correlated, PageRank also considers where citations originate and how influential the citing papers are within the network. This explains why some papers obtain a substantially better PageRank position than their citation count alone would suggest. The candidate scientific gems identified in the experiment are examples of this effect. However, their interpretation must remain cautious, since structural importance does not necessarily imply scientific quality, as also indicated by the presence of an erratum and an addendum among the selected papers.

Limitations of the analysis. The HEP-TH network is smaller and more specialized than the APS citation network used in the original Scientific Gems study, so the results should be interpreted as an application of the same general idea rather than as a direct reproduction. Moreover, PageRank measures structural importance in the citation network and does not directly evaluate the scientific quality of a paper.

6 Conclusions

The experiments confirmed that the sparse matrix-free power iteration computes PageRank accurately while avoiding the construction of the dense Google matrix. The damping factor was found to affect convergence much more strongly than the overall ranking, while the application to the HEP-TH network showed that PageRank captures structural information that is not contained in citation count alone. The dynamic update experiment further showed that previous computations can be exploited to reduce the cost of recomputing PageRank after changes in the graph, without modifying the final PageRank model.

Overall, PageRank provides a clear example of how ideas from graph theory, Markov chains, eigenvalue problems, and sparse numerical computation can be combined in a single practical algorithm. Its application to citation networks also illustrates both the usefulness and the limitations of using network structure to evaluate importance.

References

[1] P. Chen, H. Xie, S. Maslov, and S. Redner. Finding scientific gems with googleâ€™s pagerank algorithm. *Journal of Informetrics*, 1(1):8–15, None 2007.

[2] A. N. Langville and C. D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, 2006.

[3] A. Peretti and A. Roveda. On the mathematical background of google pagerank algorithm. Working Papers 25/2014, University of Verona, Department of Economics, Dec 2014.

[4] Stanford Network Analysis Project. High-Energy Physics Theory Citation Network. `https://snap.stanford.edu/data/cit-HepTh.html`. Accessed: 2026-07-21.